

262BLCH9X2

Assignment 7

Kgothlang Wesi (220113838)
Makungu Hlongwane (220029167)
Thapelo Lekgoathi (201578477)

Table of Contents

Part A.....	2
Introduction	2
Node simulations (nodes A, B and C).....	2
Creation and resolving a fork.....	3
Fork Creation	3
Resolving the Fork.....	3
Validation before comparison, longest vs heaviest	4
Edge Cases	4
Ties	4
Delayed block arrival	5
Confirmation depth (R10 000 payment)	6
Part B.....	7
Part C.....	8
References	9

Part A

Introduction

It was showed previously that a single hash-linked chain is tamper-evident, that is, any node can recompute the hashes and detect an edit/tampering. A chain that verifies on one machine is not yet a **shared** ledger. In peer-to-peer networks, every node mines and stores its own copy and messages take time to travel. This means that two honest nodes can each find a perfectly valid block at the same height. The network then holds two competing histories. **Consensus** is the rule that lets every node, looking at the same set of valid chains, independently choose the same one without needing a central coordinator.

We set out to perform the following:

- Simulate 3 nodes (A, B, C) that can hold divergent chains, both in-process and as separate operating-system processes.
- Create a fork (which is two valid competing blocks on the same parent) and resolve it with the longest-chain rule and its cumulative-work generalisation (heaviest work).
- We'll then document the edge cases, i.e., the ties, delayed block arrival and what confirmation depth should mean for a R10 000 payment.

Key concepts used throughout the work:

- **Node, tip and height:** A **node** is a peer holding its own view of the chain. Its latest block is the **tip**, the tip's index is the node's **height**.
- **Fork:** Two or more valid blocks that reference the same parent. The last block the branches share is the **fork point** (last common ancestor block).
- **Fork-choice rule:** A function that maps a set of valid chains to one canonical chain. Under the longest-chain rule, the chain with most blocks wins. Under heaviest work, the chain with the greatest cumulative proof-of-work wins (Nakamoto, 2008).
- **Orphan (stale) block:** A block that was valid when mined but is not on the canonical chain after resolution. It is not an invalid block but simply a lost one.
- **Reorganisation:** A node abandons its current tip for a heavier chain that does not contain it. Blocks after the fork point are rolled back and their transactions become unconfirmed again.
- **Confirmation depth:** If a transaction sits in the block at height h and the tip is at height H , its depth is given by $k = H - h + 1$. Since reversal remains possible but becomes less probable as k grows/ increases, the Nakamoto consensus offers only probabilistic finality.

Node simulations (nodes A, B and C)

A block is a dictionary whose hash is SHA-256 over the canonical JSON (sort_keys=True) of its index, timestamp, transactions, previous hash and nonce. The `mine_block()` function searches for nonces until the hash has D leading hexadecimal zeroes. The Difficulty used for mining is D=2 so that the computation is faster. The function `make_genesis()` uses a fixed timestamp so that all the nodes derive the same genesis block, this is important because without it, the nodes would share no history and thus not converge.

Each node is defined by the function `Node(name, chain)`, which is an object with `tip()`, `height()`, `mine()`, and `receive()` (`receive()` applies the fork-choice rule to an incoming chain and reports whether a reorganization occurred). Each node stores a deep copy of its chain such that divergence is real. A block mined by A doesn't exist for B until it's delivered explicitly.

```

== 1. Three nodes share one genesis ==
A: height=0 tip=004d4b767c... valid=True work=2
B: height=0 tip=004d4b767c... valid=True work=2
C: height=0 tip=004d4b767c... valid=True work=2
Shared tip? True

```

All three nodes start from the same genesis hash but hold separate chain objects. When A locally mines a block, only A's height will move to 1, B and C heights will stay at 0 until the block is delivered to those blocks. This is the prerequisite for a fork, each node has its own view which can diverge.

As a bonus, the network mode runs A, B and C as **separate processes** that mine concurrently and exchange blocks with a simulated network delay, so nodes genuinely race and hear about blocks late

The key aspect that follows is validation before comparison. Here `verify_chain()` function recomputes every hash, it checks the proof-of-work PoW prefix, checks that each `previous_hash` points to its parent and finally checks sequential indices. Only the chains that pass may compete.

Creation and resolving a fork

Fork Creation

We start from the genesis block, which is shared, A mines a block where Abigail pays R10 000 to Thapelo (pay-thapelo) while simultaneously B mines a block where Abigail pays the same R10 000 to Kgothalang (pay-kgothalang). C has not heard of either block and thus remains at genesis.

```

= 2. Create a fork: A and B mine competing block-1s ==
A: height=1 tip=00ed8f75fc... valid=True work=4
B: height=1 tip=00d0abf70d... valid=True work=4
C: height=0 tip=004d4b767c... valid=True work=2
A and B tips differ?      True
Both chains valid?        True True
Same parent (genesis)?    True
Fork point (last common h): 0
A block 1 tx: pay-thapelo | B block 1 tx: pay-kgothalang

```

Both chains passed the `verify_chain` criteria and both block-1s point to the same genesis hash. This means that the fork point is height 0. Both histories obey every required rule but they are in conflict in the sense that Abigail has spent the same R10 000 twice (paid it to Thapelo and Kgothalang), which is a double-spend. The network requires an extra rule to agree on one of them.

Resolving the Fork

The resolve-fork applies four steps in a fixed order:

- Discard/ remove every chain that fails the `verify_chain()` step/check
- Keep chains with maximum `chain_work`
- If more than one chain remains, then apply the documented tie-break
- Return a deep copy of the winner. `Sync_nodes()` function resolves once across all nodes and gives each node its own copy of the winner.

The two block-1s are at equal work, meaning that the fork is a tie. We let A find the next block and then synchronise:

```
= 3. Resolve: extend A's branch, then sync all nodes ==
Before sync:
  A: height=2 tip=00b58fa379... valid=True work=6
  B: height=1 tip=00d0abf70d... valid=True work=4
  C: height=0 tip=004d4b767c... valid=True work=2
After sync:
  A: height=2 tip=00b58fa379... valid=True work=6
  B: height=2 tip=00b58fa379... valid=True work=6
  C: height=2 tip=00b58fa379... valid=True work=6
All tips equal? True
Canonical block-1 tx: pay-thapelo
Orphaned (stale) blocks: [(1, '00d0abf70d')]
Orphan still valid alone? True
confirmations(pay-kgothalang) on canonical = 0
```

All of the three nodes converge on the tip of A at height 2. B then undergoes a reorganisation and its block now becomes an orphan. The above output confirms that the orphan is still a valid extension of the genesis block, it lost and didn't fail.

As a result, pay-kgothalang has zero confirmations on the canonical chain. There was no deliberate reversal of Kgothalang's payment, it was dropped out of the shared ledger.

Validation before comparison, longest vs heaviest

To test the rule, we forge the amount in A's block 1 and add four freshly mined blocks on top to produce a six-block chain. This is the longest chain offered but it fails verifications and B's 2-block chain will win. A longer invalid chain should never win a fork choice.

```
= 4. Validate before compare; longest vs heaviest ==
Tampered chain: length=6 valid=False
Winner: length=2 tx=pay-kgothalang (tampered chain rejected)
long-easy : length=5 work=6 valid=True
short-hard: length=3 work=10 valid=True
Longest-chain rule picks : long-easy
Heaviest-work rule picks : short-hard
```

We then compare 2 scoring rules when the difficulty is different/ varies. Under the longest-chain, a long chain of easy blocks (with Difficulty $D=1$) wins over a short chain with hard blocks ($D=4$) but loses under heaviest-work. Heaviest-work is the rule which matches Nakamoto's security argument, the work makes rewriting history too costly and not the number of blocks in the chain. Counting blocks alone would let an attacker win by using many cheap and low-difficulty blocks. Under uniform difficulty D , work is simply $n \times D$, so both rules agree,

Edge Cases

Ties

After the fork, A and B have equal work and so maximizing work leaves two candidates. The policy used is that the lexicographically smaller tip hash will win.

```

== 5. Edge case: a tie at equal work ==
work(A)=4 work(B)=4 -> tie
A tip: 00ed8f75fc5adcba | B tip: 00d0abf70d19d776
Winner (A,B order): 00d0abf70d19d776 pay-kgothalang
Winner (B,A order): 00d0abf70d19d776 pay-kgothalang
Order-independent? True
First-seen policy: C1 keeps pay-thapelo | C2 keeps pay-kgothalang -> views stay split
B mines block 2 -> winner: pay-kgothalang (work 6 vs 4) - tie-break no longer used

```

This strategy is deterministic and order-independent (each node which sees both chains will pick the same one regardless of arrival order). As above, it picks B's block so pay-kgothalang would be canonical briefly. In practice, Bitcoin Core uses the first-seen block. It keeps whichever chain with equal work that was received first. The First-seen criteria rewards speed and cannot be gamed. Each policy has a weakness. A hash-based tie-break lets the late arriving block displace an earlier one and this can be exploited by smart miners. The first-seen policy tolerates temporary disagreement.

In summary, a tie is resolved by the next block and not by the tie-break. Once each branch is extended it strictly has more work. The tie-break favoured pay-kgothalang but once A found block 2, pay-thapelo won. A payment with one confirmation can be reversed easily.

Delayed block arrival

A network shows latency if nodes learn of blocks in different orders. We do this for node C which first observes B's short chain and then A's heavier chain and finally receives B's stale block for the second time.

```

== 6. Edge case: delayed block arrival and reorg at node C ==
t0 C: height=0 tip=004d4b767c... valid=True work=2
t1 C hears B first -> C: height=1 tip=00d0abf70d... valid=True work=4 | reorg: True
    C sees pay-kgothalang with k = 1
t2 A's heavier chain arrives -> C: height=2 tip=00b58fa379... valid=True work=6 | reorg: True
    k(pay-kgothalang) = 0 | k(pay-thapelo) = 2
t3 stale B chain re-delivered -> reorg: False (ignored, lower work)

```

At t_1 C adopts B's branch and from C's point of view, pay-kgothalang has only one confirmation. At t_2 A's heavier chain arrives and C reorganizes. The pay-kgothalang drops to $k = 0$ and pay-thapelo has $k = 2$. At t_3 the stale chain is ignored since carries less work, late or duplicated messages are harmless. The same rule applied to different arrival orders produces temporarily different local views that then converge.

A separate-process simulation shows that A and B share a fast link (0.005 s) while C is further (0.6 s away from both A and B). The block times in the simulation are under a second and thus latency is large relative to block time.

```

Separate-process simulation: 10s mining + 2s quiet, difficulty=4
Link latency: A<->B 0.05s, A<->C 0.60s, B<->C 0.60s | messages delivered: 144
A: mined=29 height=44 tip=00005f4cb9... reorgs= 0 stale_blocks_seen=28 valid=True
B: mined=18 height=44 tip=00005f4cb9... reorgs= 2 stale_blocks_seen=28 valid=True
C: mined=25 height=44 tip=00005f4cb9... reorgs=18 stale_blocks_seen=28 valid=True
Converged on one tip after quiet period? True
Canonical blocks by miner: {'A': 29, 'B': 15, 'C': 0} | total mined: 72 | orphaned: 28

```

All three nodes converge on one valid tip after mining stops and this confirms that the fork-choice rule works under concurrency. 31 of the 80 blocks were orphaned and the poorly connected node C had only 3 canonical blocks and the most reorganisations. This is because it kept mining on stale tips. Stale rates rise

with the ratio of network delay to block time. Bitcoin's 10-minute blocks keep its rate below 1%. Latency also favours well-connected miners.

Confirmation depth (R10 000 payment)

Confirmation depth seeks to answer the question: “how much work would someone have to redo to remove this payment?”. With $k = H - h + 1$, pay-thapelo at $h = 1$ reaches $k = 6$ when the tip is at $H = 6$. The function `settled_for_policy()` maps each policy to a single parameter k :

```
= 7. Confirmation depth k = H - h + 1 for the R10 000 payment ==
tip H=1: k=1  coffee(k>=1)=True  policy(k>=3)=False  OTC(k>=6)=False
tip H=2: k=2  coffee(k>=1)=True  policy(k>=3)=False  OTC(k>=6)=False
tip H=3: k=3  coffee(k>=1)=True  policy(k>=3)=True   OTC(k>=6)=False
tip H=4: k=4  coffee(k>=1)=True  policy(k>=3)=True   OTC(k>=6)=False
tip H=5: k=5  coffee(k>=1)=True  policy(k>=3)=True   OTC(k>=6)=False
tip H=6: k=6  coffee(k>=1)=True  policy(k>=3)=True   OTC(k>=6)=True
Attacker catch-up probability P(q, k) [Nakamoto 2008, s.11]
  k  q=0.1  q=0.2  q=0.3
0  1.0000  1.0000  1.0000
1  0.2046  0.4159  0.6277
2  0.0510  0.2039  0.4457
3  0.0132  0.1032  0.3246
6  0.0002  0.0143  0.1321
```

For a selection of k , reversal is deemed a credit risk. If an attacker controls a share q of the hash rate then Nakamoto defines a probability $P(q, k)$ that the attacker will ever catch up from k blocks behind.

Expected loss on R10 000	k = 1	k = 3	k = 6
q = 10%	R2 046	R132	R2.43
q = 20%	R4 159	R1 032	R143
q = 30%	R6 277	R3 246	R1 321

K is a risk parameter, loss probability falls with depth but never reaches zero and thus finality is probabilistic. Given a 0.1% tolerance of value (R10) and a 10% attacker, the smallest acceptable depth is $k=5$ and thus the conventional $k= 6$ is recommended. This is approximately an hour on Bitcoin. $K=1$ would suffice for low-value payments.

The formula also assumes an honest majority, as q tends to 50% no depth is actually safe. The depth must be read with the cost of the work. On Bitcoin, a single block reward (3.125 BTC) would exceed R10 000 so a reorganisation for that amount is irrational. For small chains with rentable hash power, deep reorganisations have occurred (e.g. Bitcoin Gold) and so far more confirmations are needed. On our chain ($D = 2$) any depth can be rewritten in milliseconds and thus it only illustrates the mechanism not the security.

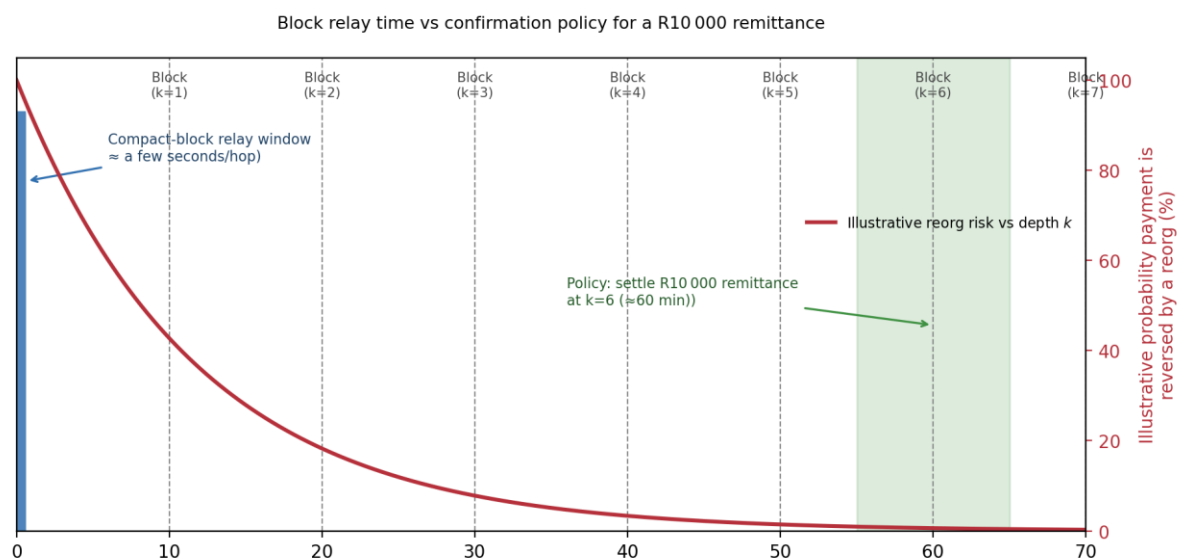
Part B

Chapter 10: The Bitcoin Network

Central Mechanism: Compact Block Relay(BIP152)

The Compact Block Relay is designed to optimize how new blocks are announced and transmitted across the flat peer-to-peer(P2P) mesh network. When a miner discovers a new block, transmission delays across the global network create temporary forks, which are known as block-finding races. Larger mining pools hold an unfair advantage during these races because their internal nodes learn about the newly mined blocks faster than smaller, isolated miners. This latency advantage threatens Bitcoin's essential Property of decentralisation(Page 219). Full nodes that relay unconfirmed transactions already maintain those transactions in their local mempools. When a new block is mined, re-transmitting the full body of transaction data is redundant. BIP152 solves this by replacing the full transaction payloads with 6-byte short transaction identifiers, reconstructing the block locally and requesting only what's missing. This reaches 97–99% bandwidth efficiency without shrinking blocks themselves; it only cuts redundant transfer. Two modes trade off speed against redundancy: low-bandwidth (default) announces only a header hash and waits to be asked; high-bandwidth pushes compact block pre-validation to about 3 peers, checking only for valid proof-of-work (Page 220).

Figure 1. Block Relay Time vs Confirmation Policy for Remittance



Consider Thabo, working abroad, who sends a R10,000 remittance home to his family in South Africa via Bitcoin. His transaction is included in a block at height h . The receiving family wants to know how long to wait before treating the payment as safely settled. The blue bar marks the compact-block relay window, a few seconds per hop during which Thabo's block reaches the network. Relative to the rest of the axis, scaled in minutes, this window is effectively instantaneous, showing that propagation delay operates on a seconds-scale clock, distinct from confirmation depth. The red curve, read against the right axis, sketches an illustrative decay in reorganisation risk as confirmation depth k increases, with dashed lines marking successive blocks at the roughly ten-minute average interval. The shaded region marks the desk's policy: Thabo's family releases or spends the funds only once $k = 6$ confirmations (approximately sixty minutes) have accumulated, following the conventional benchmark for high-value Bitcoin payments. The figure shows that relay speed governs how often forks arise, while confirmation depth governs residual risk once one occurs.

Part C

Confirmation depth for cross-border retail payments

We recommend that the receiving provider wait for six confirmations before paying the recipient in local currency, for an illustrative R10 000 Bitcoin remittance. However, this is not a guarantee that the payment cannot be reversed, it is a starting policy. The provider should also confirm that the transaction is still on the longest chain accepted by the network and that no network disruption or unusual fork is being investigated before making the payout.

When a payment receives its first confirmation when it is included in the block, each block added after that increases its confirmation depth. This is computed as $k = H - h + 1$, where the payment of the block height is h and the current tip height is H . According to the explanation by Decker and Wattenhofer (2013), blocks take time to propagate through the Bitcoin network, and miners may build on different valid blocks during that delay. A reorganisation can be used to remove the payment from the accepted chain, if the branch that has the payment later loses. The recipients can wait for more confirmations which lowers this risk, although it also means that they wait longer for their money.

Therefore, six confirmations should not be treated as a fixed rule for every R10 000 transfer; rather, it should be treated as a policy that provides a review against real network conditions and their own loss tolerance. If the payment is made before a reversal, when the exchange rate is fixed and when the recipient's local payment becomes irrevocable, the recipients should agree amongst themselves who bears the loss. The receiving provider should pause the payout and check the transaction again if a competing chain appears while the payment is still pending.

The Practical Byzantine Fault Tolerance by Castro and Liskov's (1999) provides a different way to reach an agreement. It states that instead of relying on an increasing number of proof-of-work confirmations, known validators can agree on the order of transactions under specified fault assumptions. This can indicate or provide a clearer point of ledger finality; it also depends on how validators are governed and chosen. No matter which option the providers choose or use, they must still set a clear rule for when a payment becomes final. That is, to the exact moment when the money set to the recipient in their local currency can no longer be reversed or taken back. In other words, providers must define payout finality separately.

References

- Castro, M. and Liskov, B. (1999) ‘Practical Byzantine fault tolerance’, *Proceedings of the Third Symposium on Operating Systems Design and Implementation*, pp. 173–186.
- Decker, C. and Wattenhofer, R. (2013) ‘Information propagation in the Bitcoin network’, *2013 IEEE Thirteenth International Conference on Peer-to-Peer Computing*, pp. 1–10.
<https://doi.org/10.1109/P2P.2013.6688704>